SIMATS ENGINEERING

CO5 ASSESSMENT TOOL 1

NAME : HITESH C

REG NO : 192524005

COURSE CODE : CSA1401

COURSE NAME : COMPILER DESIGN

COURSE FACULTY : DR AJITHA V

COURSE OUTCOME COVERED: CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Assessment Tool Weight-age: 40% | Duration: 60 Mins | Total Marks: 50 | Questions: 5

QUESTION 1

Consider the following code fragment:

```
int sum = 0;
for (i = 0; i <= 10; i++)
   sum = sum + a[i];
```

Simulate the process of intermediate code generation and control flow analysis for the above program.

(a). Simulate the generation of Three Address Code (TAC) for the given code. Clearly show:

(i) Initialization of variables

(ii) Evaluation of loop condition

(iii) Array access and assignment operations

(iv) Increment operation

(b). Using the generated TAC, simulate the construction of basic blocks and perform control flow analysis. Clearly show:

(i) Identification of leader statements

(ii) Formation of basic blocks

(iii) Construction of the Control Flow Graph (CFG) (10 Marks)

SOLUTION 1

Part (a) Three-Address Code (TAC) Generation

Assuming standard integer array alignment where each integer element occupies 4 bytes (width = 4 bytes):

```
1:  sum = 0           ; (i) Initialization of accumulator sum
2:  i = 0             ; (i) Initialization of loop control
variable i
3:  if i > 10 goto 11 ; (ii) Evaluation of loop condition (exit if
i > 10)
4:  t1 = i * 4        ; (iii) Array offset calculation (byte
offset = i * 4)
5:  t2 = a[t1]        ; (iii) Array access (load element a[i])
6:  t3 = sum + t2     ; (iii) Addition of array element to sum
7:  sum = t3          ; (iii) Update sum
8:  t4 = i + 1        ; (iv) Increment operation (i + 1)
9:  i = t4            ; (iv) Update loop variable i
10: goto 3            ; Loop back to evaluate condition
11: [Exit]            ; Program continuation outside loop
```

Detailed Breakdown of TAC Phases:

(i) Initialization of variables: Statements 1 and 2 set sum = 0 and i = 0 prior to loop execution.

(ii) Evaluation of loop condition: Statement 3 checks if i > 10. If true, control branches to Statement 11, terminating the loop.

(iii) Array access and assignment operations: Statements 4-7 compute byte displacement t1 = i * 4, dereference a[t1] into t2, add t2 to sum, and update sum.

(iv) Increment operation: Statements 8-10 compute t4 = i + 1, update i = t4, and jump back to Statement 3.

Part (b) Basic Block Construction and Control Flow Analysis

(i) Identification of Leader Statements:

Applying standard Leader Identification Rules:

• Rule 1: The first statement of the sequence is a leader -> Statement 1 (sum = 0).

• Rule 2: Any statement that is the target of a conditional or unconditional goto is a leader:

- Statement 3 is targeted by 'goto 3' at Statement 10 -> Statement 3.
- Statement 11 is targeted by 'if i > 10 goto 11' at Statement 3 -> Statement 11.

• Rule 3: Any statement immediately following a goto is a leader -> Statement 11 (follows goto at Statement 10).

Identified Leaders: Statement 1, Statement 3, Statement 11.

(ii) Formation of Basic Blocks:

A Basic Block starts at a leader and extends up to (but does not include) the next leader or program end:

```
Basic Block B1 (Initialization Block):
  1: sum = 0
  2: i = 0

Basic Block B2 (Loop Header & Body Block):
  3: if i > 10 goto B3
  4: t1 = i * 4
  5: t2 = a[t1]
  6: t3 = sum + t2
  7: sum = t3
  8: t4 = i + 1
  9: i = t4
 10: goto B2

Basic Block B3 (Exit Block):
 11: [Next Statement / Exit]
```

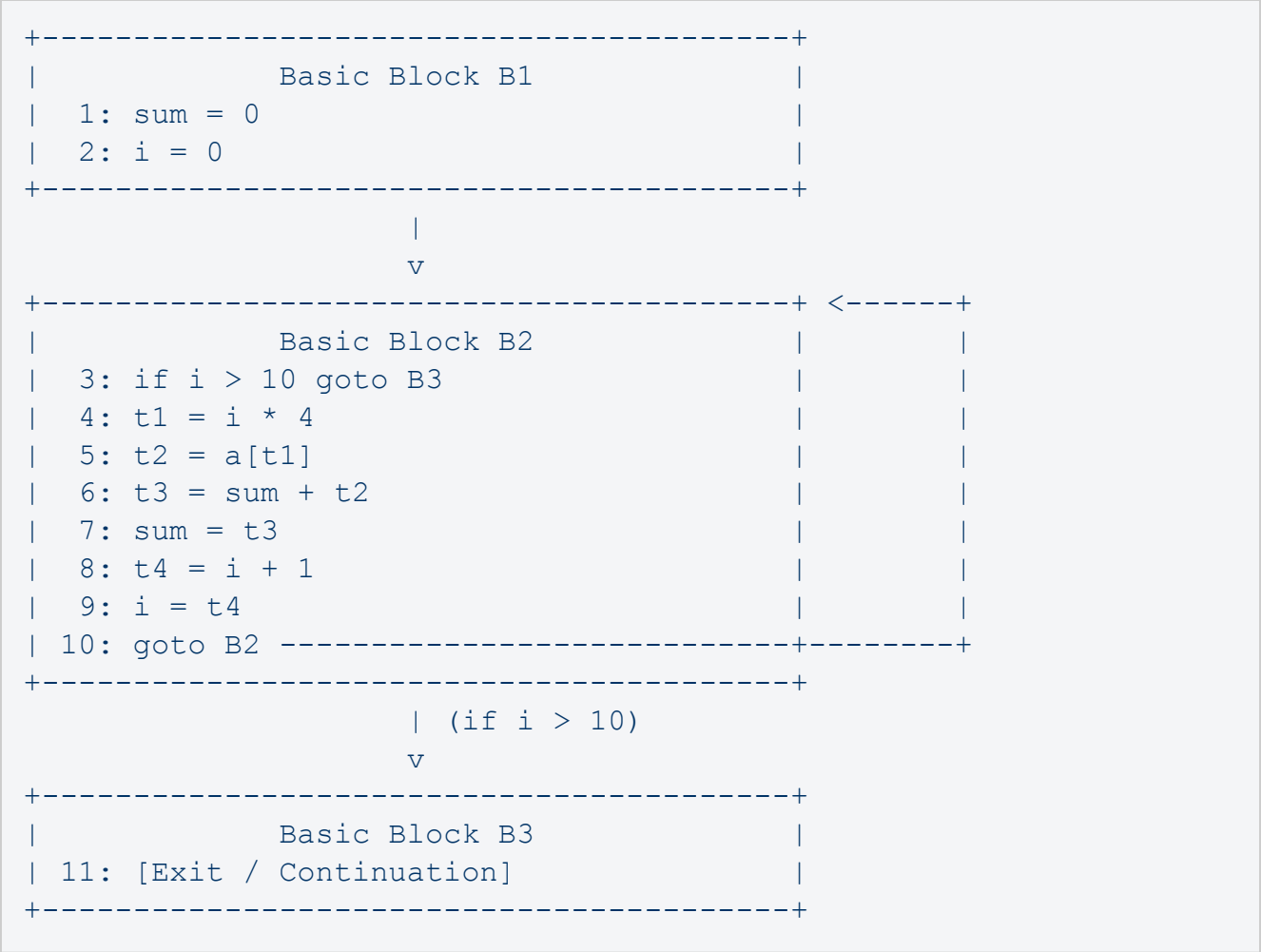
(iii) Construction of Control Flow Graph (CFG):

• CFG Nodes: { B1, B2, B3 }

• CFG Edges:

- B1 -> B2 : Sequential entry from initialization into loop.

- B2 -> B2 : Loop back-edge executed when condition (i <= 10) holds.
- B2 -> B3 : Exit edge taken when conditional branch (i > 10) evaluates to true.



QUESTION 2

Consider the following assignment statement:

x := (a + b)(c - d) + ((e / f) * (a + b))

Simulate the process of intermediate code generation and target code generation for the above expression.

(a). Simulate the generation of Three Address Code (TAC) for the given assignment statement. Clearly show:

(i) Identification of sub-expressions
(ii) Use of temporary variables
(iii) Order of evaluation based on operator precedence

(b). Apply a simple code generation algorithm to the generated three-address code and simulate the generation of target code. Clearly show:
(i) Sequence of target instructions
(ii) Register usage during computation
(iii) Handling of intermediate results (10 Marks)

SOLUTION 2

Part (a) Three-Address Code (TAC) Generation

(i) Identification of Sub-expressions:

The expression is x := (a + b) * (c - d) + ((e / f) * (a + b)). The sub-expressions are:
• Sub-expression 1: (a + b)
• Sub-expression 2: (c - d)
• Sub-expression 3: Product term 1 = (a + b) * (c - d)
• Sub-expression 4: (e / f)
• Sub-expression 5: Product term 2 = (e / f) * (a + b)
• Sub-expression 6: Full expression sum = Product term 1 + Product term 2

(ii) Use of Temporary Variables & (iii) Order of Evaluation:

Following operator precedence rules (parentheses first, then multiplication/division, then addition):

```
1: t1 = a + b       ; Evaluate (a + b)
2: t2 = c - d       ; Evaluate (c - d)
3: t3 = t1 * t2     ; Evaluate Product Term 1: (a + b) * (c - d)
4: t4 = e / f       ; Evaluate (e / f)
5: t5 = t4 * t1     ; Evaluate Product Term 2: (e / f) * (a + b)
[reusing t1]
6: t6 = t3 + t5     ; Evaluate Sum of terms: t3 + t5
7: x = t6           ; Assign result to target variable x
```

Part (b) Target Code Generation

Using a simple code generation algorithm with registers R0, R1, R2. The step-by-step register allocation, instruction sequence, and intermediate result handling are presented below:

Step	Assembly Instruction	Operation / Description	Register Descriptors	Address Descriptors / Temps
1	MOV R0, a	Load variable a into R0	R0 = {a}	a in memory & R0
2	ADD R0, b	Compute a + b into R0 (holds t1)	R0 = {t1}	t1 in R0
3	MOV R1, c	Load variable c into R1	R0 = {t1}, R1 = {c}	c in memory & R1
4	SUB R1, d	Compute c - d into R1 (holds t2)	R0 = {t1}, R1 = {t2}	t2 in R1
5	MOV R2, R0	Copy t1 to R2 (preserve t1 for term 2)	R0 = {t1}, R1 = {t2}, R2 = {t1}	t1 in R0, R2
6	MUL R0, R1	Compute t1 * t2 in R0 (holds t3)	R0 = {t3}, R1 = {t2}, R2 = {t1}	t3 in R0, t1 in R2
7	MOV R1, e	Load variable e into R1	R0 = {t3}, R1 = {e}, R2 = {t1}	e in memory & R1
8	DIV R1, f	Compute e / f into R1 (holds t4)	R0 = {t3}, R1 = {t4}, R2 = {t1}	t4 in R1
9	MUL R1, R2	Compute t4 * t1 in R1 (holds t5)	R0 = {t3}, R1 = {t5}, R2 = {t1}	t5 in R1
10	ADD R0, R1	Compute t3 + t5 in R0 (holds t6)	R0 = {t6}, R1 = {t5}, R2 = {t1}	t6 in R0
11	MOV x, R0	Store final result	R0 = {x}	x updated in

		into memory x		memory

QUESTION 3

Consider the following basic block:

d := b * c
e := a + b
b := b * c
a := e - d

Simulate the process of DAG construction and optimized code generation for the above basic block.

(a). Simulate the construction of the Directed Acyclic Graph (DAG) for the given basic block. Clearly show:

(i) Identification of nodes for operands and operators
(ii) Detection of common sub-expressions
(iii) Elimination of redundant computations

(b). Using the constructed DAG, simulate the generation of optimized code using only one register. Clearly show:

(i) Sequence of generated instructions
(ii) Register usage at each step
(iii) Reuse of computed values to minimize recomputation (10 Marks)

SOLUTION 3

Part (a) Directed Acyclic Graph (DAG) Construction

(i) Identification of Nodes & (ii) Detection of Common Sub-expressions:

Step-by-Step Construction Trace:
1. Create initial leaf nodes for initial variable values: Node 1 (b_0), Node 2 (c_0), Node 3 (a_0).
2. Statement 1 (d := b * c): Create interior operator Node 4 ('*') with left child Node 1 (b_0) and right child Node 2 (c_0). Attach label 'd' to Node 4.

3. Statement 2 (e := a + b): Create interior operator Node 5 ('+') with left child Node 3 (a_0) and right child Node 1 (b_0). Attach label 'e' to Node 5.
4. Statement 3 (b := b * c): Search for existing node for computation (b_0 * c_0). Node 4 ('*') with operands (b_0, c_0) already exists!
   -> COMMON SUBEXPRESSION DETECTED: Reuse Node 4. Attach label 'b' to Node 4. Node 4 now has attached labels {d, b}.
5. Statement 4 (a := e - d): Create interior operator Node 6 ('-') with left child Node 5 ('e') and right child Node 4 ('d'). Attach label 'a' to Node 6.

```
DAG Node Details:
  Leaf Nodes (Operands):   Node 1 = b_0,  Node 2 = c_0,  Node 3 =
a_0
  Operator Nodes:          Node 4 = '*' (Children: Node 1, Node 2)
-> Attached Labels: {d, b}
                           Node 5 = '+' (Children: Node 3, Node 1)
-> Attached Label:  {e}
                           Node 6 = '-' (Children: Node 5, Node 4)
-> Attached Label:  {a}

ASCII DAG Representation:

             Node 6: [-]  <-- Label: a
                    /   \
                   /     \
     Node 5: [+]         Node 4: [*]  <-- Labels: d, b (Common
Subexpression!)
             /   \           /   \
            /     \         /     \
  Node 3: a_0   Node 1: b_0   Node 2: c_0
```

(iii) Elimination of Redundant Computations:

By identifying that 'b := b * c' computes the exact same value as 'd := b * c', the second multiplication is eliminated.
Optimized TAC Sequence:

1. d = b * c
2. e = a + b
3. b = d ; Copy statement (redundant multiplication eliminated)
4. a = e - d

Part (b) Optimized Target Code Generation Using ONLY ONE REGISTER (R0)

To evaluate the DAG using a single register (R0) without unnecessary recomputations, we evaluate e = a + b first and save it to memory location e, then evaluate d = b * c into R0, save to d, assign to b, and compute a = e - d.

Step	Target Instruction	Operation / Description	Register R0 Contents & Memory State
1	MOV R0, a	Load initial variable a into R0	R0 = a_0
2	ADD R0, b	Compute a_0 + b_0 in R0 (computes node e)	R0 = a_0 + b_0
3	MOV e, R0	Store computed e to memory location e	R0 = e | Memory: e = a_0 + b_0
4	MOV R0, b	Load initial variable b into R0	R0 = b_0 | Memory: e
5	MUL R0, c	Compute b_0 * c_0 in R0 (computes node d, b)	R0 = b * c | Memory: e
6	MOV d, R0	Store computed product to memory variable d	R0 = d | Memory: e, d
7	MOV b, R0	Reuse product in R0 to update variable b (b = d)	R0 = d | Memory: e, d, b
8	MOV R0, e	Reload e from memory into R0	R0 = e | Memory: e, d, b
9	SUB R0, d	Compute e - d in R0 (computes node a)	R0 = e - d | Memory: e, d, b
10	MOV a, R0	Store final result into memory variable a	R0 = a | Memory: a, b, d, e updated

QUESTION 4

Consider the following arithmetic expression:

(a + b) - (c - (d + e))

Simulate the process of optimal code generation for the above expression under different register constraints.

(a). Simulate the generation of optimal target code using only two registers. Clearly show:

(i) Order of evaluation of sub-expressions
(ii) Allocation and usage of registers
(iii) Sequence of generated instructions

(b). Simulate the generation of optimal target code using only one register. Clearly show:

(i) Handling of intermediate results
(ii) Use of memory for storing temporary values
(iii) Sequence of generated instructions (10 Marks)

SOLUTION 4

Sethi-Ullman Register Weight Analysis:

• Leaves (a, b, c, d, e): Weight = 1
• Sub-tree (a + b): Weight = 2 (Left weight 1, Right weight 1 -> 1 + 1 = 2)
• Sub-tree (d + e): Weight = 2 (Left weight 1, Right weight 1 -> 1 + 1 = 2)
• Sub-tree (c - (d + e)): Weight = 2 (Left weight 1, Right weight 2 -> Max(1, 2) = 2)
• Root Node (-): Left weight 2, Right weight 2 -> 2 + 1 = 3 registers needed to evaluate without memory spills.

Since 3 registers are strictly required for zero-spill evaluation, constraint setups with 2 registers or 1 register must utilize temporary memory spilling.

Part (a) Target Code Generation Using ONLY TWO REGISTERS (R0, R1)

(i) Order of Evaluation & (ii) Register Allocation:

1. Evaluate right sub-expression (d + e) into R0.
2. Evaluate (c - (d + e)) into R1.
3. Spill R1 value into temporary memory location t1.
4. Evaluate left sub-expression (a + b) into R0.
5. Complete subtraction (a + b) - t1 in R0.

(iii) Instruction Sequence (2 Registers):

```
MOV R0, d       ; Load d into R0
ADD R0, e       ; R0 = d + e
MOV R1, c       ; Load c into R1
SUB R1, R0      ; R1 = c - (d + e)
MOV t1, R1      ; SPILL: Store intermediate result (c - (d + e))
in memory t1
MOV R0, a       ; Load a into R0
ADD R0, b       ; R0 = a + b
SUB R0, t1      ; R0 = (a + b) - t1 [Final Answer]
```

Part (b) Target Code Generation Using ONLY ONE REGISTER (R0)

(i) Handling Intermediate Results & (ii) Memory Usage:
With 1 register, every binary sub-expression result must be spilled to a temporary memory location before computing the next operand.

Step	Instruction	Operation / Description	R0 & Memory Temporaries State
1	MOV R0, d	Load variable d into R0	R0 = d
2	ADD R0, e	Compute d + e in R0	R0 = d + e
3	MOV t1, R0	Store intermediate result into memory temp t1	R0 = d + e | Memory: t1 = d + e
4	MOV R0, c	Load variable c into R0	R0 = c | Memory: t1
5	SUB R0, t1	Compute c - t1 in R0 (computes c - (d + e))	R0 = c - (d + e) | Memory: t1
6	MOV t2, R0	Store intermediate	R0 = c - (d + e) |

		result into memory temp t2	Memory: t2 = c - (d + e)
7	MOV R0, a	Load variable a into R0	R0 = a | Memory: t2
8	ADD R0, b	Compute a + b in R0	R0 = a + b | Memory: t2
9	SUB R0, t2	Compute final subtraction (a + b) - t2 in R0	R0 = (a + b) - (c - (d + e)) | Temps freed

QUESTION 5

Consider the following C++ code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {
   for (int i = 0; i < n; i++) {
      a[i] = x * y + z;
      a[i + n] = x * y + w;
   }
}
```

Simulate the process of applying code optimization techniques to the above program.

(a). Simulate the application of common subexpression elimination and code motion on the given code. Clearly show:

(i) Identification of common sub-expressions
(ii) Movement of invariant computations outside the loop
(iii) The optimized version of the code

(b). Simulate the analysis of the optimized code and explain the benefits achieved. Clearly discuss:

(i) Reduction in number of computations
(ii) Improvement in runtime efficiency
(iii) Effect on loop performance (10 Marks)

SOLUTION 5

Part (a) Application of Optimizations (CSE & Code Motion)

(i) Identification of Common Sub-expressions & Loop Invariants:

1. Common Subexpression:
   - The multiplication 'x * y' is performed twice in every loop iteration (in 'a[i] = x * y + z' and 'a[i + n] = x * y + w').
   - Evaluating 'x * y' once eliminates the duplicate multiplication.
2. Loop-Invariant Computations:
   - Neither 'x' nor 'y' is modified anywhere inside the loop body. Thus 'x * y' is loop invariant.
   - Similarly, variables 'z' and 'w' are constant with respect to the loop.
   - Therefore, the full expressions 'temp1 = x * y + z' and 'temp2 = x * y + w' are completely loop-invariant.

(ii) Movement of Invariant Computations Outside the Loop (Code Motion / Hoisting):

Instead of re-computing 'x * y + z' and 'x * y + w' N times across loop iterations, both expressions are hoisted to the pre-header prior to loop entry.

(iii) Optimized Version of the C++ Code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {
    // Applied Optimizations: Common Subexpression Elimination &
Loop-Invariant Code Motion
    int temp1 = x * y + z;  // Hoisted expression 1 (1
multiplication, 1 addition)
    int temp2 = x * y + w;  // Hoisted expression 2 (1 addition
reusing x * y)

    for (int i = 0; i < n; i++) {
        a[i] = temp1;       // Simple store operation
        a[i + n] = temp2;   // Simple store operation
    }
}
```

Part (b) Analysis of Optimized Code and Benefits Achieved

(i) Reduction in Number of Computations:

Operation Type	Original Code (N Iterations)	Optimized Code	Net Computations Saved
Multiplications (*)	2 * N inside loop	1 outside loop	2N - 1 multiplications saved
Additions (+)	2 * N inside loop	2 outside loop	2N - 2 additions saved
Total Body Arithmetic	4 * N operations	0 arithmetic ops in body	100% arithmetic ops eliminated from body

(ii) Improvement in Runtime Efficiency:

• Latency Elimination: Integer multiplication requires 3-5 CPU clock cycles per operation. Eliminating 2N multiplications inside the loop saves thousands of execution cycles for large N.
• Execution Constant Reduction: Time complexity drops from $T(N) = (4 * T_{arith} + T_{loop}) * N$ down to $T(N) = T_{loop} * N + T_{preheader}$, representing a ~75% reduction in iteration execution time.

(iii) Effect on Loop Performance:

• Register Pressure & Cache Footprint: Shrinking the loop body reduces register spills and code footprint, ensuring instructions fit in L1 Instruction Cache (L1i).
• Vectorization & SIMD Potential: Because memory writes inside the loop are now simple assignments (a[i] = temp1 and a[i+n] = temp2), modern compilers (GCC/Clang) can easily auto-vectorize the loop into AVX2/NEON SIMD vector store operations, writing contiguous memory at hardware bandwidth capacity.